

© Joshua M. Willman 2020

J. M. Willman, *Beginning PyQt*

https://doi-org.ezproxy.sfpl.org/10.1007/978-1-4842-5857-6_4

4. Learning About Layout Management

Joshua M. Willman¹

(1) Hampton, VA, USA

The previous chapters laid the foundation for getting started in PyQt5 as you learned to create GUIs with more functionality by adding widgets such as `QPushButton` and `QCheckBox`. Rather than continue to barrage you with PyQt's numerous widgets, taking a moment to learn about the various layout managers will save us some trouble moving forward.

Layout management is the way in which we arrange widgets in our windows. This can involve a number of different factors, including size and position, resize handling, and adding or removing widgets. Layout management is also very important to consider for the user looking at your application. Do a quick image search on the Internet for “worst GUI layouts” and you will see numerous applications crammed with widgets with no clear reasoning.

A **layout manager** is a class that contains methods which we can use to arrange widgets inside windows. They are useful for communicating between child and parent widgets to make them utilize the space in a window more efficiently.

In this chapter we are going to take a look at four methods that can be used for layout management in PyQt:

1. **Absolute positioning** with `move()`.
2. `QBoxLayout` which is useful for creating simple GUIs with horizontal or vertical layouts.
3. `QFormLayout` is a convenience layout useful for making application forms.
4. `QGridLayout` allows for more control over arranging widgets by specifying x and y coordinate values.

This chapter will also cover the idea of nesting layouts for creating more elaborate applications.

We will also take a look at a few new widgets and classes:

- `QTextEdit` – Similar to `QLineEdit` but creates a text entry field with more space
- `QFileDialog` – Native file dialog of PyQt that allows the user to open or save files
- `QButtonGroup` – To organize push button and checkbox widgets

- `QSpinBox` – A text box that displays integer values that the user can cycle through
- `QComboBox` – Presents a list of options in a dropdown-style widget

To make things simpler to understand in this chapter, let's first take a look at a few key concepts for using layout managers.

Choosing a Layout Manager

Setting the layout manager or changing the one we want to use in our applications isn't very difficult to do.

When you import modules in the beginning of your program, be sure to also include the layout manager(s) you want to use like so:

```
from PyQt5.QtWidgets import (QApplication, QWidget, QHBoxLayout, QFormLayout)
```

Here we import both the `QHBoxLayout` and `QFormLayout` classes from `QtWidgets`. Then to set a specific layout manager inside a parent window or widget, we first must create an instance of that layout manager. In the following code, we call `QVBoxLayout` and then set the layout within the parent window using `setLayout()`.

```
v_box = QVBoxLayout()
parent_window.setLayout(v_box)
```

Using a layout manager isn't necessary, but it is definitely preferred in order to make your applications easier to organize and to rearrange if necessary.

Customizing the Layout

So you've got your layout manager chosen. You've created your widgets. How do you go about adding them to the window?

```
name = QTextEdit()
v_box.addWidget(name)
```

In this code, we create a `QTextEdit` widget and then call the `addWidget()` method to add it in the layout. Be sure to consider what type of layout manager the parent window or widget is using before adding a child widget. For each layout manager, some of the parameters may change or you may need to call a different method to add widgets, but the concept is the same – create a widget, then add that widget into your layout. We will go over more details when we get to each specific manager.

If you need to create a more complex application with widgets arranged horizontally, vertically, or maybe even arranged in a grid, it is also possible to nest layouts in PyQt. **Nesting layouts** involves placing one layout manager inside of another. This can be accomplished by calling the `addLayout()` method and passing the name of a layout as a parameter.

One of the great things about using a layout manager is that when you resize the windows, the widgets in the window will all adjust accordingly.

However, each layout manager has its own way of determining the spacing, alignment, size, or border around the widgets. These can all be manipulated and we will look at a few of these methods in the upcoming projects.

Absolute Positioning – Move()

While many people will recommend using layout managers, you can of course create layouts without them. This idea is called **absolute positioning** and it involves specifying the size and position values for each widget. This is the method we used in the previous chapters when we used **move()** to arrange widgets.

If you do decide to use absolute positioning, there are a few drawbacks to keep in mind. First of all, resizing the main window will not cause the widgets in it to adjust their size or position. Something else to keep in mind is the differences between operating systems, such as fonts and font sizes which could drastically change the look and layout of our GUI.

Absolute positioning can be most useful for setting the position and size values of widgets that are contained within other widgets.

Project 4.1 – Basic Notepad GUI

For our first project, let's take a look at creating a simple interface, a notepad GUI, to demonstrate how to use absolute positioning. A notepad is a way to capture our ideas or to take notes. It generally starts off blank and we fill in the information line by line. The benefit of having a digital notepad is that we can input and edit text much more easily than with real paper. Electronic notepads don't just include a blank area to write, but also tools which can be found at the top of the GUI window to help open, save, or edit notes.

This project, as can be seen in Figure [4-1](#), lays the foundation for our notepad GUI. In Chapter [5](#), we will take a look at how to improve upon this example by creating a menu interface and adding editing tools.

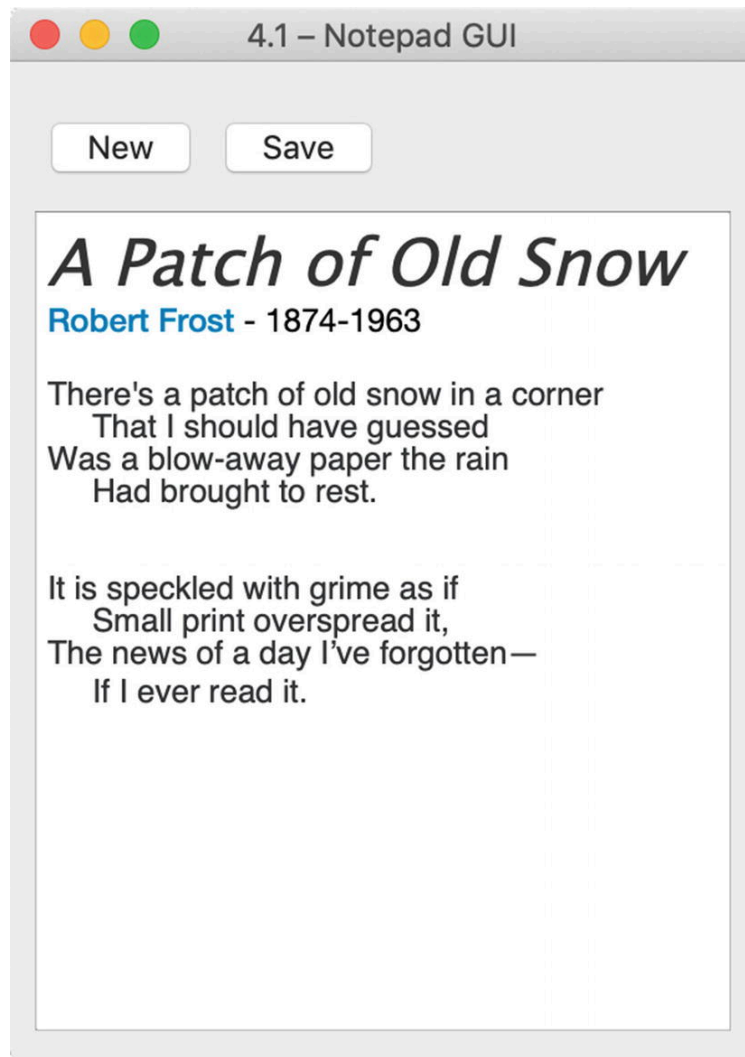


Figure 4-1 Basic notepad GUI

The QTextEdit Widget

If we are going to create a notepad GUI, then we need a text entry field that will allow us to enter and edit more than one line of text at a time.

The **QTextEdit** widget allows a user to enter text, either plain or rich text, and permits editing such as copy, paste, or cut. The widget can handle characters or paragraphs of text. Paragraphs are simply long strings that are word-wrapped into the widget and end with a newline character. QTextEdit is also useful for displaying lists, images, and tables or providing an interface for displaying text using HTML.

Take a look at the Solution code to the notepad GUI to see how to create a QTextEdit widget.

The QFileDialog Class

The **QFileDialog** class can be used to select files or directories found on your computer. This can be useful for locating and opening a file or looking for a directory to save a file and giving your file a name.

To open a file, we call the `getOpenFileName()` method, set the parent, create a title for the dialog box, display contents of a specific directory, and display files matching the patterns given in the string `"All Files (*.*);;Text Files (*.txt)"`. You can also display image or other file types.

```
file_name = QFileDialog.getOpenFileName(self, 'Open File', "/Users/user_name/Desktop/", "All Files (*.*);;
```

Saving a file is done in a similar fashion.

```
file_name = QFileDialog.getSaveFileName(self, 'Save File', "/Users/user_name/Desktop/", "All Files (*.*);;
```

The look of the dialog box that appears will also reflect the type of system you are using. To change these properties, you could access `QFileDialog.Options()` and alter the dialog properties and appearance.

```
options = QFileDialog.Options()
options = QFileDialog.DontUseNativeDialog # By default native dialog is used
```

Basic Notepad GUI Solution

For this project, the GUI will consist of three widgets, two `QPushButton`s and a `QTextEdit` field (Listing 4-1). Users will be able to select the new button to clear the text in the line edit field or save the text to a file by clicking the save button and opening a dialog box.

```
# notepad.py
# Import necessary modules
import sys
from PyQt5.QtWidgets import (QApplication, QWidget, QPushButton, QTextEdit, QMessageBox, QFileDialog)
class Notepad(QWidget):
    def __init__(self): # constructor
        super().__init__()
        self.initializeUI()
    def initializeUI(self):
        """
        Initialize the window and display its contents to the screen
        """
        self.setGeometry(100, 100, 300, 400)
        self.setWindowTitle('4.1 - Notepad GUI')
        self.notepadWidgets()
        self.show()
    def notepadWidgets(self):
        """
        Create widgets for notepad GUI and arrange them in window
        """
        # Create push buttons for editing menu
        new_button = QPushButton("New", self)
        new_button.move(10, 20)
        new_button.clicked.connect(self.clearText)
        save_button = QPushButton("Save", self)
        save_button.move(80, 20)
        save_button.clicked.connect(self.saveText)
        # Create text edit field
        self.text_field = QTextEdit(self)
        self.text_field.resize(280, 330)
        self.text_field.move(10, 60)
    def clearText(self):
```

```

"""
    If the new button is clicked

, display dialog asking user if they want to clear the text edit field or not.
"""
    answer = QMessageBox.question(self, "Clear Text",
        "Do you want to clear the text?", QMessageBox.No | QMessageBox.Yes,
        QMessageBox.Yes)
    if answer == QMessageBox.Yes:
        self.text_field.clear()
    else:
        pass
def saveText(self):
    """
    If the save button is clicked, display dialog to save the text in the text edit field to a text
    """
    options = QFileDialog.Options()
    notepad_text = self.text_field.toPlainText()
    file_name, _ = QFileDialog.getSaveFileName(self, 'Save File', "", "All Files (*);;Text Files (*.
    if file_name:
        with open(file_name, 'w') as f:
            f.write(notepad_text)
if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = Notepad()
    sys.exit(app.exec_())

```

Listing 4-1 Code for creating notepad GUI

Our notepad can be seen in Figure 4.1. The text displayed in the widgets shows that the QLineEdit widget can support different fonts, colors, and text sizes.

Explanation

When you use absolute positioning, you can think of the window as a grid where the top-left corner has the x and y coordinates (0, 0). If you create a window with height equal to 100 and width also 100, then the bottom-right corner has values (99, 99). To arrange widgets using `move()`, you need to specify values within the height and width range.

For example, the `new_button` push button is created and positioned as follows:

```

new_button = QPushButton("New", self)
new_button.move(10, 20) # x = 10, y = 20

```

Three widgets are created in Listing 4.1, two buttons and a `QTextEdit` widget for inputting text. We can use the buttons to either clear text or save the text we have typed.

When the user clicks the `save_button`, the `saveText()` method is called which displays a `QFileDialog` like the one shown in Figure 4.2.

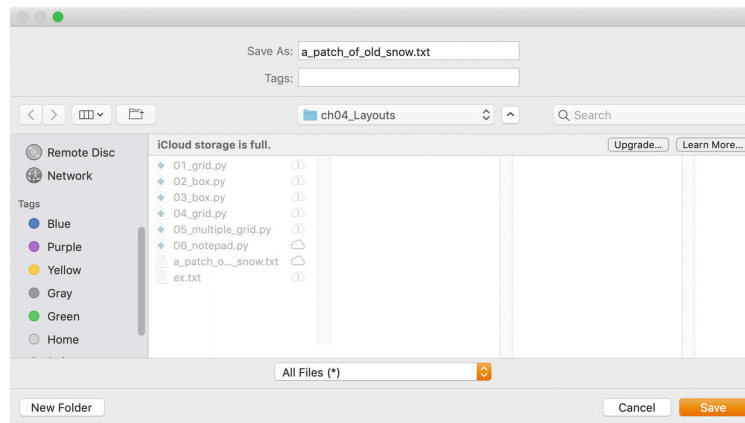


Figure 4-2 QFileDialog box that opens to save the text from the notepad GUI

The contents of the text file are shown in Figure 4-3. Because we save the file as a text file, it loses some information but still retains the spacing and paragraphs separated by newlines. To keep the rich text information, you could save the file using HTML. This will be covered in Chapter 5.

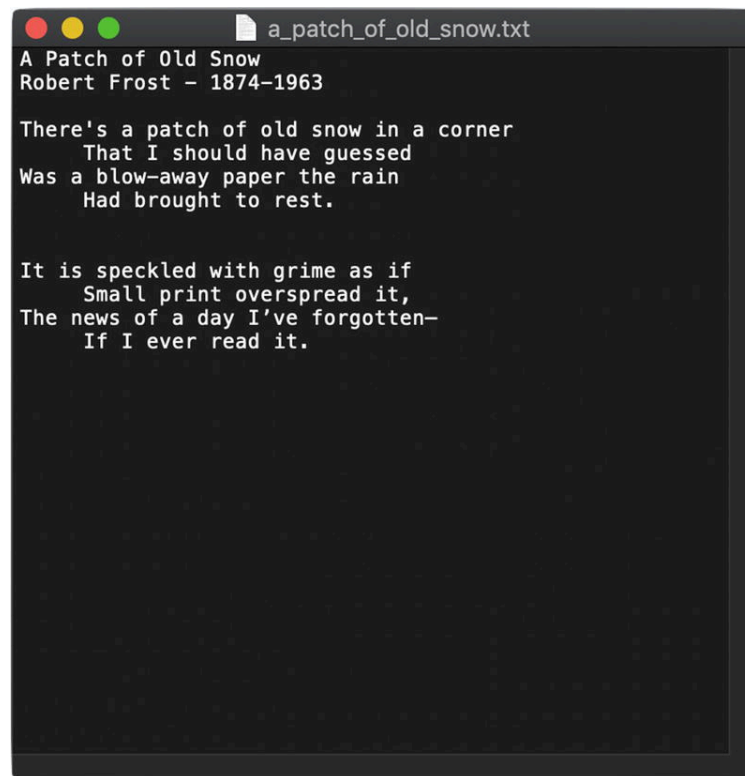


Figure 4-3 Text file showing the text saved from notepad GUI

The QHBoxLayout and QVBoxLayout Classes

Arranging widgets can be accomplished easily with the QBoxLayout classes. PyQt has two different QBoxLayout styles, QHBoxLayout and QVBoxLayout:

- **QHBoxLayout** – Used to arrange widgets horizontally from left to right in the window
- **QVBoxLayout** – Used to arrange widgets vertically from top to bottom in the window

Creating a basic GUI with only one of these layout managers is possible, but the real potential comes from being able to combine the two of them

to create more elaborate layouts. Together we can combine them, along with the `addStretch()` method, to place widgets anywhere in the window. Think of `addStretch()` as an adjustable blank space that can be used to help arrange widgets relative to each other or to help place widgets in the window.

In the following project, we will take a look at how to use `QHBoxLayout` and `QVBoxLayout` in the same program to create a simple survey GUI application.

Project 4.2 – Survey GUI

Creating a survey to collect data from users can be very useful for businesses or for research. In the following project, we will take a look at how to use the `QBoxLayout` class to create a simple window that displays a question to the user and allows them to select an answer.

From personal experience, the Python language is very good at automating repetitive tasks. In university, I needed to collect data from almost a thousand participants in order to research marketing trends related to how they spent money at sporting events. For my research I decided to create an application that would ask the user a question and then store their answers in a Python list. When a participant reached the end of the survey, their answers were written to a file. This greatly helped later when I needed to use that same data to create graphs and charts.

Figure 4-4 shows the program we are going to make in Project 4.2.

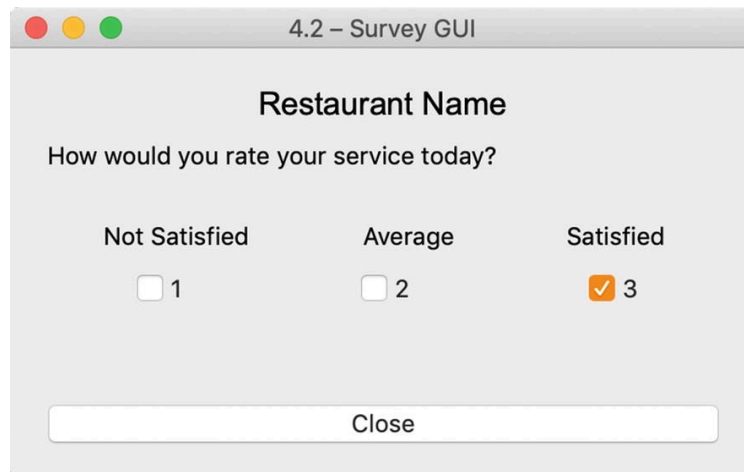


Figure 4-4 Survey GUI

The `QButtonGroup` Class

You may often have a few checkboxes or buttons that need to be grouped together to make it easier to manage them. Luckily, PyQt has the `QButtonGroup` class to help not only group and arrange buttons together, but also has the ability to make buttons mutually exclusive. This is also helpful if you only want one checkbox to be checked at a time.

`QButtonGroup` is not actually a widget, but a container where you can add widgets. Therefore, you can't actually add `QButtonGroup` to a layout. The

following code shows the method of how to import and set up QButtonGroup in your application:

```
from PyQt5.QtWidgets import QButtonGroup, QCheckBox
b_group = QButtonGroup() # Create instance of QButtonGroup
# Create two checkboxes
cb_1 = QCheckBox("CB 1")
cb_2 = QCheckBox("CB 2")
# Add checkboxes into QButtonGroup
b_group.addButton(cb_1)
b_group.addButton(cb_2)
# Connect all buttons in a group to one signal
b_group.buttonClicked.connect(cbClicked)
def cbClicked(cb):
    print(cb)
```

In the above code we create two QCheckBox widgets and add them to the QButtonGroup using the `addButton()` method. To make the buttons mutually exclusive, we check to see if a signal is sent not from each individual button but from the button group instead. This is done with

```
b_group.buttonClicked.connect(cbClicked)
```

Survey GUI Solution

The survey GUI consists of QLabel widgets to display the title, question, and ratings labels for each checkbox. For the checkboxes in the window, the text beside each label could have been left blank, but the numbers are left as a visual cue to the user. Once the user selects a choice, they can close the window using a QPushButton (Listing [4-2](#)).

```
# survey.py
# Import necessary modules
import sys
from PyQt5.QtWidgets import (QApplication, QWidget, QLabel, QPushButton, QCheckBox, QButtonGroup, QHBoxLayout)
from PyQt5.QtGui import QFont

class DisplaySurvey(QWidget):
    def __init__(self):
        super().__init__()
        self.initializeUI()
    def initializeUI(self):
        """
        Initialize the window and display its contents to the screen
        """
        self.setGeometry(100, 100, 400, 230)
        self.setWindowTitle('4.2 - Survey GUI')
        self.displayWidgets()
        self.show()
    def displayWidgets(self):
        """
        Set up widgets using QHBoxLayout and QVBoxLayout.
        """
        # Create label and button widgets
        title = QLabel("Restaurant Name")
        title.setFont(QFont('Arial', 17))
        question = QLabel("How would you rate your service today?")
        # Create horizontal layouts
```

```

title_h_box = QHBoxLayout()
title_h_box.addStretch()
title_h_box.addWidget(title)
title_h_box.addStretch()
ratings = ["Not Satisfied", "Average", "Satisfied"]
# Create checkboxes and add them to horizontal layout, and add stretchable
# space on both sides of the widgets
ratings_h_box = QHBoxLayout()
ratings_h_box.setSpacing(60) # Set spacing between in widgets in horizontal layout
ratings_h_box.addStretch()
for rating in ratings:
    rate_label = QLabel(rating, self)
    ratings_h_box.addWidget(rate_label)
ratings_h_box.addStretch()
cb_h_box = QHBoxLayout()
cb_h_box.setSpacing(100) # Set spacing between in widgets in horizontal layout
# Create button group to contain checkboxes
scale_bg = QButtonGroup(self)
cb_h_box.addStretch()
for cb in range(len(ratings)):
    scale_cb = QCheckBox(str(cb), self)
    cb_h_box.addWidget(scale_cb)
    scale_bg.addButton(scale_cb)
cb_h_box.addStretch()
# Check for signal when checkbox is clicked
scale_bg.buttonClicked.connect(self.checkboxClicked)
close_button = QPushButton("Close", self)
close_button.clicked.connect(self.close)
# Create vertical layout and add widgets and h_box layouts
v_box = QVBoxLayout()
v_box.addLayout(title_h_box)
v_box.addWidget(question)
v_box.addStretch(1)
v_box.addLayout(ratings_h_box)
v_box.addLayout(cb_h_box)
v_box.addStretch(2)
v_box.addWidget(close_button)
# Set main layout of the window

self.setLayout(v_box)
def checkboxClicked(self, cb):
    """
    Print the text of checkbox selected.
    """
    print("{} Selected.".format(cb.text()))
if __name__ == '__main__':
    app = QApplication(sys.argv)
    window = DisplaySurvey()
    sys.exit(app.exec_())

```

Listing 4-2 Code for creating survey GUI

The survey GUI can be seen in Figure [4-4](#).

Explanation

After importing all of the PyQt classes and setting up the `DisplaySurvey` class, we begin by creating some labels, setting up the text for the ratings labels, and creating the `close_button`.

The application consists of three separate `QHBoxLayout` objects – `title_h_box`, `ratings_h_box`, and `cb_h_box` – and a single `QVBoxLayout` layout, `v_box`. For this GUI, `v_box` will act as the container for all of the other widgets and layouts, arranged vertically from top to bottom.

Combining Box Layouts and Arranging Widgets

When we say combining layouts, what that really means is nesting one type of box layout inside of another type to get the benefit of vertical or horizontal layouts.

The following bit of code shows how to create a `QHBoxLayout` object and add a widget to it:

```
# Create horizontal layouts
title_h_box = QHBoxLayout()
title_h_box.addStretch()
title_h_box.addWidget(title)
title_h_box.addStretch()
```

The `addStretch()` method acts like an invisible widget that can be used to help arrange widgets in a layout manager. Widgets in `QHBoxLayout` are organized left to right, so in `title_h_box`, `addStretch` is added to the left, `title` in the middle, and another `addStretch` to the right. This centers the `title` in `title_h_box`.

To add the rating labels and checkboxes to the window, a separate `QHBoxLayout` is created for each one. Each widget added is spaced out using the `setSpacing()` method, which is useful for creating a fixed amount of space between widgets inside of a layout.

Adding layouts or widgets to a parent layout is as simple as changing the method called.

```
v_box = QVBoxLayout() # Create vertical layout
v_box.addLayout(title_h_box) # Add horizontal layout
v_box.addWidget(question) # Add widget
```

The QFormLayout Class

In Chapter 3 we looked at how to make a create new user GUI (Project 3.2) that would collect a user's information. In that project, each line consisted of a `QLabel` widget on the left and a `QLineEdit` widget on the right. They were then arranged in the window using absolute positioning.

For situations like this where you need to create a form to collect information from a user, PyQt provides the **QFormLayout** class. It is a layout class that arranges its children widgets into a two-column layout, the left column consisting of labels and the right one consisting of entry field widgets such as `QLineEdit` or `QSpinBox`. The `QFormLayout` class makes designing these kinds of GUIs very convenient.

Project 4.3 – Application Form GUI

We all have to fill out application forms at some point, applying for a job, when you want to go to university, trying to get insurance for your car, or signing up for a new bank account.

For this project let's take a look at creating an application form that someone could use to set up an appointment at the hospital like in Figure 4-5. When filling out an electronic application, you can combine many different widgets to not only reduce the size of the window but also minimize the amount of clutter from text that you might usually see on a paper application.

Before getting started on the application form, we should learn about two new widgets – QSpinBox and QComboBox – that we will use in the application GUI.

The screenshot shows a window titled "4.3 – Application Form GUI" containing an "Appointment Submission Form". The form has the following elements:

- Full Name:** A text input field with an orange border.
- Address:** A text input field.
- Mobile Number:** A text input field with two dashes as placeholders.
- Age:** A QSpinBox widget showing the value 0.
- Height:** A text input field with "cm" as a unit.
- Weight:** A text input field with "kg" as a unit.
- Gender:** A QComboBox widget showing "Male".
- Past Surgeries:** A text input field with the placeholder text "separate by ' '".
- Blood Type:** A QComboBox widget showing "A".
- Desired Time:** Three QSpinBox/QComboBox widgets showing "0", ":00", and "AM".
- Submit Appointment:** A large button at the bottom.

Figure 4-5 Application form GUI

The QSpinBox and QComboBox Widgets

Rather than using a QLineEdit widget for the user to input information, sometimes you may want them to only be allowed to select from a list of predetermined values. Both QSpinBox and QComboBox widgets are very useful for handling this kind of task.

QSpinBox creates an object that is similar to a text box, but allows the user to select integer values either by typing a value into the widget or by clicking the up and down arrows. You can also edit the range of the values, set the step size when the arrow is clicked, set a starting value, or even add prefixes or suffixes in the box. There are also other kinds of spin boxes in PyQt, such as **QDateEdit**, to select date and time values.

QComboBox is a way to display a list of options for the user to select from. When a user clicks the arrow button, a pop-up list appears and displays a collection of possible selections.

In Listing 4-3 we will take a look at how to create both kinds of objects, add them to our layout, and find out how to use the values in **QSpinBox** to update other widgets in the GUI window.

```
# spin_combo_boxes.py
# Import necessary modules
import sys
from PyQt5.QtWidgets import (QApplication, QWidget, QLabel, QComboBox, QSpinBox, QHBoxLayout, QVBoxLayout)
from PyQt5.QtGui import QFont
from PyQt5.QtCore import Qt
class SelectItems(QWidget):
    def __init__(self):
        super().__init__()
        self.initializeUI()
    def initializeUI(self):
        """
        Initialize the window and display its contents to the screen
        """
        self.setGeometry(100, 100, 300, 200)
        self.setWindowTitle('ComboBox and SpinBox')
        self.itemsAndPrices()
        self.show()
    def itemsAndPrices(self):
        """
        Create the widgets so users can select an item from the combo boxes
        and a price from the spin boxes
        """
        info_label = QLabel("Select 2 items you had for lunch and their prices.")
        info_label.setFont(QFont('Arial', 16))
        info_label.setAlignment(Qt.AlignCenter)
        self.display_total_label = QLabel("Total Spent: $")
        self.display_total_label.setFont(QFont('Arial', 16))
        self.display_total_label.setAlignment(Qt.AlignRight)
        # Create list of food items and add those items

to two separate combo boxes
        lunch_list = ["egg", "turkey sandwich", "ham sandwich", "cheese", "hummus", "yogurt", "apple",
        lunch_cb1 = QComboBox()
        lunch_cb1.addItem(lunch_list)
        lunch_cb2 = QComboBox()
        lunch_cb2.addItem(lunch_list)
        # Create two separate price spin boxes
        self.price_sb1 = QSpinBox()
        self.price_sb1.setRange(0,100)
        self.price_sb1.setPrefix("$")
        self.price_sb1.valueChanged.connect(self.calculateTotal)
        self.price_sb2 = QSpinBox()
```

```

self.price_sb2.setRange(0,100)
self.price_sb2.setPrefix("$")
self.price_sb2.valueChanged.connect(self.calculateTotal)
# Create horizontal boxes to hold combo boxes and spin boxes
h_box1 = QHBoxLayout()
h_box2 = QHBoxLayout()
h_box1.addWidget(lunch_cb1)
h_box1.addWidget(self.price_sb1)
h_box2.addWidget(lunch_cb2)
h_box2.addWidget(self.price_sb2)
# Add widgets and layouts to QVBoxLayout
v_box = QVBoxLayout()
v_box.addWidget(info_label)
v_box.addLayout(h_box1)
v_box.addLayout(h_box2)
v_box.addWidget(self.display_total_label)
self.setLayout(v_box)
def calculateTotal(self):
    """
    Calculate and display total price from spin boxes and change value shown in QLabel
    """
    total = self.price_sb1.value() + self.price_sb2.value()
    self.display_total_label.setText("Total Spent: ${}".format(str(total)))
if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = SelectItems()
    sys.exit(app.exec_())

```

Listing 4-3 Code for creating QSpinBox and QComboBox widgets

Your window should look similar to the one seen in Figure 4-6.

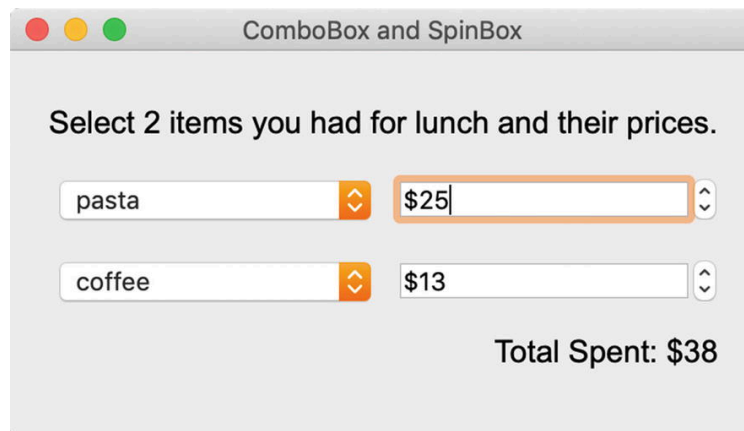


Figure 4-6 GUI to show how to create QSpinBox and QComboBox widgets

Explanation

The code for this application is also another demonstration of how to create nested layouts in PyQt. Here we create two instances of `QHBoxLayout` and add them to a vertical layout.

We create two separate combo boxes, `lunch_cb1` and `lunch_cb2`, and add the list of items that we want to be displayed to each of them using the `addItem()` method. Then two separate spin boxes are created, `price_sb1` and `price_sb2`.

```
self.price_sb1.setRange(0,100)
```

```
self.price_sb1.setPrefix("$")
```

The `setRange()` method is used to set the upper and lower boundaries for a spin box and `setPrefix()` can be used to display other text inside of the text box, in this case a dollar sign. This can be helpful to give the user more information about the widget's purpose. All of these widgets are then added to two separate horizontal layouts, `h_box1` and `h_box2`.

Note Since the two `QComboBox` objects and the two `QSpinBox` objects each contain the same values, you may have the urge to just try and use them over again when adding them to `QVBoxLayout`. This won't work. When you add an item to a widget or to a layout, that widget takes ownership of the item. This means you cannot add an item to more than one widget or layout. You will need to create a new instance.

Finally, as we change the values in the spin boxes, they both send a signal that is connected to the `calculateTotal()` method. This will dynamically update the value for `display_total_label` in the window.

Application Form GUI Solution

The application form GUI consists of a number of different widgets, including `QLabel`, `QLineEdit`, `QSpinBox`, `QComboBox`, `QTextEdit`, and `QPushButton` (Listing 4-4). Nesting layouts is also possible with the `QFormLayout` manager.

```
# application.py
# Import necessary modules
import sys
from PyQt5.QtWidgets import (QApplication, QWidget, QLabel, QPushButton, QFormLayout, QLineEdit, QTextE
from PyQt5.QtGui import QFont
from PyQt5.QtCore import Qt
class GetApptForm(QWidget):
    def __init__(self):
        super().__init__()
        self.initializeUI()
    def initializeUI(self):
        """
        Initialize the window and display its contents to the screen
        """
        self.setGeometry(100, 100, 300, 400)
        self.setWindowTitle('4.3 - Application Form GUI')
        self.formWidgets()
        self.show()
    def formWidgets(self):
        """
        Create widgets that will be used in the application form.
        """
        # Create widgets
        title = QLabel("Appointment Submission Form")
        title.setFont(QFont('Arial', 18))
        title.setAlignment(Qt.AlignCenter)
        name = QLineEdit()
        name.resize(100, 100)
        address = QLineEdit()
        mobile_num = QLineEdit()
        mobile_num.setInputMask("000-000-0000;")
        age_label = QLabel("Age")
        age = QSpinBox()
        age.setRange(1, 110)
```

```

height_label = QLabel("Height")
height = QLineEdit()
height.setPlaceholderText("cm")
weight_label = QLabel("Weight")
weight = QLineEdit()
weight.setPlaceholderText("kg")
gender = QComboBox()
gender.addItem("Male", "Female")
surgery = QTextEdit()
surgery.setPlaceholderText("separate by ','")
blood_type = QComboBox()
blood_type.addItem("A", "B", "AB", "O")
hours = QSpinBox()
hours.setRange(1, 12)
minutes = QComboBox()
minutes.addItem(":00", ":15", ":30", ":45")
am_pm = QComboBox()
am_pm.addItem("AM", "PM")
submit_button = QPushButton("Submit Appointment")
submit_button.clicked.connect(self.close)
# Create horizontal layout and add age, height, and weight to h_box

h_box = QHBoxLayout()
h_box.addSpacing(10)
h_box.addWidget(age_label)
h_box.addWidget(age)
h_box.addWidget(height_label)
h_box.addWidget(height)
h_box.addWidget(weight_label)
h_box.addWidget(weight)
# Create horizontal layout and add time information
desired_time_h_box = QHBoxLayout()
desired_time_h_box.addSpacing(10)
desired_time_h_box.addWidget(hours)
desired_time_h_box.addWidget(minutes)
desired_time_h_box.addWidget(am_pm)
# Create form layout
app_form_layout = QFormLayout()
# Add all widgets to form layout
app_form_layout.addRow(title)
app_form_layout.addRow("Full Name", name)
app_form_layout.addRow("Address", address)
app_form_layout.addRow("Mobile Number", mobile_num)
app_form_layout.addRow(h_box)
app_form_layout.addRow("Gender", gender)
app_form_layout.addRow("Past Surgeries ", surgery)
app_form_layout.addRow("Blood Type", blood_type)
app_form_layout.addRow("Desired Time", desired_time_h_box)
app_form_layout.addRow(submit_button)
self.setLayout(app_form_layout)

if __name__ == '__main__':
    app = QApplication(sys.argv)
    window = GetApptForm()
    sys.exit(app.exec())

```

Listing 4-4 Code for creating application form GUI

When completed, your GUI should look similar to Figure [4-5](#).

Note Depending upon what system you are working on, the look and layout of the widgets in your window will change.

Explanation

Using the `QFormLayout` class is pretty straightforward. In the `formWidget` method, the widgets that will be used in this GUI are instantiated in the beginning. An important one to point out is the `mobile_num` line edit object.

Any type of character can naturally be typed into the `QLineEdit` entry field. However, if you want to limit the type, size, or manner in which characters can be input, then you can create an input mask by calling the `setInputMask()` method. `setInputMask()` also can be used to set the maximum number of characters.

Two parts of this application have widgets arranged horizontally. For these widgets we will add them in `QHBoxLayout` objects.

The `QFormLayout` object is created by

```
app_form_layout = QFormLayout()
```

Next, all widgets and layouts are added to the form layout using the `addRow()` method. Finally, the layout for our window is set using `self.setLayout(app_form_layout)`.

Widgets and layouts can be added to a `QFormLayout` object in the following ways:

```
form_layout.addRow(QWidget)
form_layout.addRow("text", QWidget)
form_layout.addRow(layout)
```

The first one will add a widget and may cause it to stretch to fit the window. The second fits the text and its widget into a two-column layout. The last one can be used to nest layouts.

The QGridLayout Class

The **QGridLayout** layout manager is used to arrange widgets in rows and columns similar to a spreadsheet or matrix. The layout manager takes the space within its parent window or widget and divides it up according to the sizes of the widgets within that row (or column). Adding space between widgets, creating a border, or stretching widgets across multiple rows or columns is also possible.

Understanding how to add and manipulate widgets using `QGridLayout` is also easier. The grid for the layout manager starts at value (0, 0) which is the top leftmost cell. To add a widget underneath it (the next row), simply add 1 to the left value, (1, 0). To keep moving down the rows, keep increasing the left value. To move across columns, increase the right value.

Let's take a look at how to make a to-do list using `QGridLayout`.

Project 4.4 – To-Do List GUI

We all have things that we must do every day, and many of us need a way to help organize our busy lives. For this project we will take a look at creating a basic layout for a to-do list.

Some to-do lists are broken down by hours of the day, by importance of goals, or by various other tasks we may need to do for that day, week, or even month. Once a goal is complete, we need some way to check off a task or remove it.

The project will consist of a to-do list made up of two parts, a list of things you must do on the left and daily appointments on the right. The “Must Dos” will consist of QCheckBox and QLineEdit widgets. The “Appointments” will be separated into three sections, morning, noon, and evening, and will use QTextEdit widgets to give the user an area to write down their tasks. You can see the GUI we will be building in Figure 4-7.

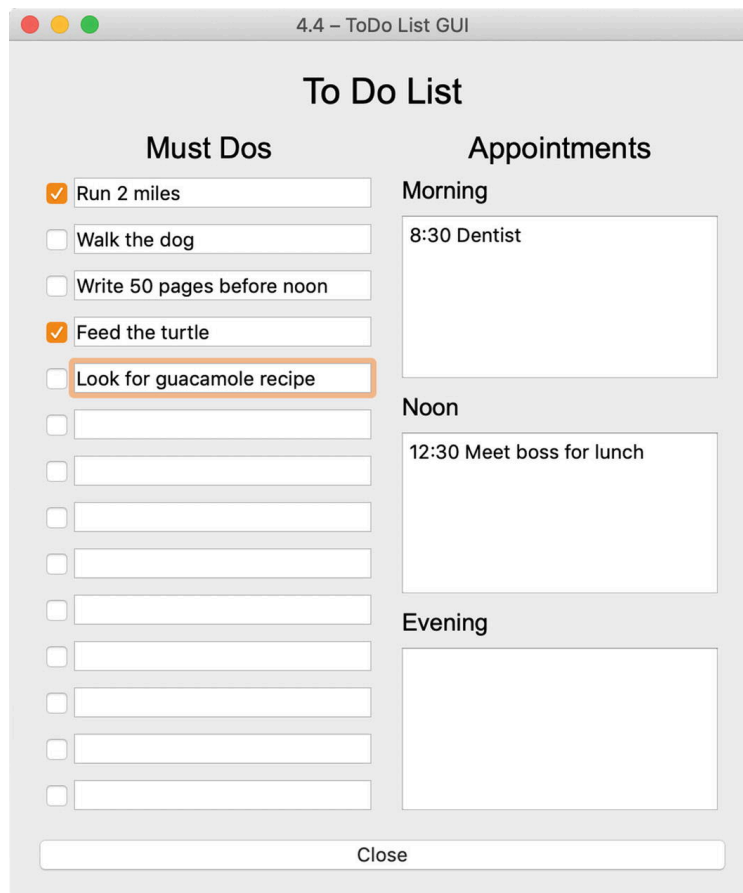


Figure 4-7 To-do list GUI

To-Do List GUI Solution

For this project (Listing 4-5), we will be focusing mainly on how to create the GUI and arrange widgets using the QGridLayout class as the main layout. This project also includes a nested QVBoxLayout for the “Appointments” layout.

```
# todo.py
# Import necessary modules
import sys
from PyQt5.QtWidgets import (QApplication, QWidget, QLabel, QTextEdit, QLineEdit, QPushButton, QCheckBox)
from PyQt5.QtGui import QFont
from PyQt5.QtCore import Qt
```

```

class ToDoList(QWidget):
    def __init__(self): # Constructor
        super().__init__()
        self.initializeUI()
    def initializeUI(self):
        """
        Initialize the window and display its contents to the screen
        """
        self.setGeometry(100, 100, 500, 350)
        self.setWindowTitle('4.4 - ToDo List GUI')
        self.setupWidgets()
        self.show()
    def setupWidgets(self):
        """
        Create widgets for to-do list GUI and arrange them in the window
        """
        # Create grid layout

        main_grid = QGridLayout()
        todo_title = QLabel("To Do List")
        todo_title.setFont(QFont('Arial', 24))
        todo_title.setAlignment(Qt.AlignCenter)
        close_button = QPushButton("Close")
        close_button.clicked.connect(self.close)
        # Create section labels for to-do list
        mustdo_label = QLabel("Must Dos")
        mustdo_label.setFont(QFont('Arial', 20))
        mustdo_label.setAlignment(Qt.AlignCenter)
        appts_label = QLabel("Appointments")
        appts_label.setFont(QFont('Arial', 20))
        appts_label.setAlignment(Qt.AlignCenter)
        # Create must-do section
        mustdo_grid = QGridLayout()
        mustdo_grid.setContentsMargins(5, 5, 5, 5)
        mustdo_grid.addWidget(mustdo_label, 0, 0, 1, 2)
        # Create checkboxes and line edit widgets
        for position in range(1, 15):
            checkbox = QCheckBox()
            checkbox.setChecked(False)
            linedit = QLineEdit()
            linedit.setMinimumWidth(200)
            mustdo_grid.addWidget(checkbox, position, 0)
            mustdo_grid.addWidget(linedit, position, 1)
        # Create labels for appointments section
        morning_label = QLabel("Morning")
        morning_label.setFont(QFont('Arial', 16))
        morning_entry = QTextEdit()
        noon_label = QLabel("Noon")
        noon_label.setFont(QFont('Arial', 16))
        noon_entry = QTextEdit()
        evening_label = QLabel("Evening")
        evening_label.setFont(QFont('Arial', 16))
        evening_entry = QTextEdit()
        # Create vertical layout and add widgets

        appt_v_box = QVBoxLayout()
        appt_v_box.setContentsMargins(5, 5, 5, 5)
        appt_v_box.addWidget(appts_label)
        appt_v_box.addWidget(morning_label)

```

```

        appt_v_box.addWidget(morning_entry)
        appt_v_box.addWidget(noon_label)
        appt_v_box.addWidget(noon_entry)
        appt_v_box.addWidget(evening_label)
        appt_v_box.addWidget(evening_entry)
        # Add other layouts to main grid layout
        main_grid.addWidget(todo_title, 0, 0, 1, 2)
        main_grid.addLayout(mustdo_grid, 1, 0)
        main_grid.addLayout(appt_v_box, 1, 1)
        main_grid.addWidget(close_button, 2, 0, 1, 2)
        self.setLayout(main_grid)

if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = ToDoList()
    sys.exit(app.exec_())

```

Listing 4-5 Code for the to-do list GUI

When you are finished, your GUI should look similar to Figure [4-7](#).

Explanation

Here we start by creating our `ToDoList` class which inherits from `QWidget`. In the `setupWidgets()` method, a few `QLabel` widgets which will serve as header labels for the GUI and the different sections are created. A `QPushButton`, which will close the program, is also instantiated.

The must-do section uses the `QGridLayout` class. A margin can be set to frame a layout using the `setContentMargins()` method.

```
mustdo_grid.setContentMargins(5, 5, 5, 5)
```

Each integer specifies the size of the border in pixels, (left, top, right, bottom).

Then the layout managers, `mustdo_grid` and `appt_v_box`, and their widgets are created and added. Finally, the title, the two layouts, and the close button are added to the `main_grid` layout, and `main_grid` is set as the main layout using `setLayout()`.

Adding Widgets and Spanning Rows and Columns with `QGridLayout`

Since widgets will be placed in a grid-like structure, when you add a new object to the layout, you must specify the row and column values as parameters of the `addWidget()` or `addLayout()` methods. Take a look at the following lines:

```

main_grid.addWidget(todo_title, 0, 0)
main_grid.addLayout(mustdo_grid, 1, 0)
main_grid.addLayout(appt_v_box, 1, 1)

```

The `todo_title` `QLabel` widget is added to the `main_grid` layout at the position where the row equals 0 and column equals 0, which is also the top-left corner. Then, the `mustdo_grid` is added directly below it by increasing the row value to 1 and leaving the column value equal to 0. Finally, we move over one column for the `appt_v_box` layout by setting the column value to 1. If you want to build a GUI with more widgets using

QGridLayout, then you would just continue in this manner moving away from 0, 0.

But what happens if you have a widget in a column or a row that is next to another widget that needs to take up more space in the vertical or horizontal direction? QGridLayout allows us to specify the number of rows or columns that we want a single widget or layout to **span**. Spanning can be thought of as stretching a widget horizontally or vertically to help us better arrange our GUI.

```
main_grid.addWidget(clear_button, 2, 0, 1, 3)
```

The extra two parameters at the end, 1 and 3, tell the layout manager that we want to span one row and three columns. This causes the widget to stretch horizontally.

Summary

Taking the time to learn about layout management will save you time and effort when coding your own GUI applications. In this chapter we reviewed absolute positioning using the `move()` method and learned about three of PyQt's layout managers – QBoxLayout, QFormLayout, and QGridLayout. Each of these classes has their own special purpose, but one of the real powers of PyQt is how convenient it is to nest them into other layouts to make more complex GUIs.

It is important to note that any of the subclasses within QWidget can also use a layout manager to manage their children. The advantages of using a layout manager include

- Positioning of child widgets
- Setting default sizes for windows
- Handling resizing of windows
- Updating content in the window or parent widget when something changes, such as type of font, font size, and hiding, showing, or removing of a child widget

You can actually design and lay out your interface graphically using the Qt Designer Application. We will take a brief look at how to do this in Chapter 7.

In Chapter 5 we are going to take a look at how to add menus to our applications.